

Data Storytelling

Matplotlib

Matplotlib is a graph plotting library in python that serves as a visualization utility. It is one of the oldest and most versatile libraries, providing low-level control over every element of a figure. Matplotlib allows basic to advanced plotting with extremely flexible and fluid, so you have no limitations on what you can create and how it looks. In some cases, you can even do a picture-in-picture type of visualization where you simultaneously display two visualizations. Matplotlib allows a user to add markers and change colors, and for some types of visualizations, you can enable interactions.

Typical Use Cases of matplotlib

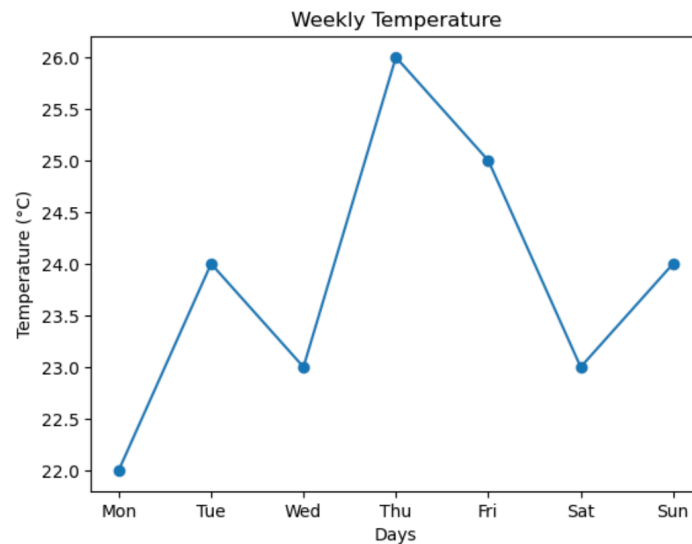
- Line charts, bar charts, histograms
- Custom figures for publication
- Embedding plots into applications

Line Chart

A line chart or line plot is a graphical representation used to show the relationship between two continuous variables by connecting data points with a straight line. It is commonly used to visualize trends, patterns or changes over time. In a line chart, the x-axis typically represents the independent variable while the y-axis represents the dependent variable.

Syntax: `plt.plot(x, y, color, linestyle, linewidth, marker)`

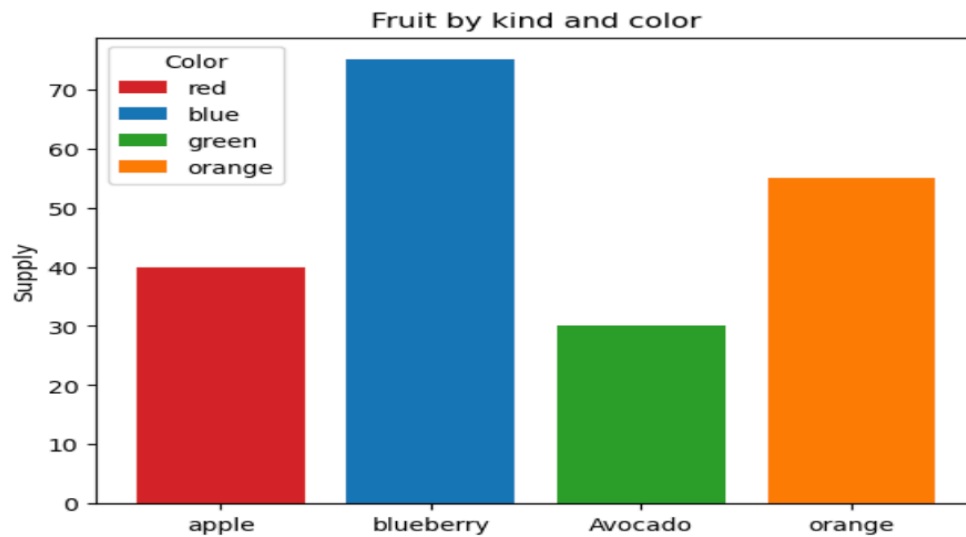
```
import matplotlib.pyplot as plt
import numpy as np
days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
temperature = [22, 24, 23, 26, 25, 23, 24]
plt.plot(days, temperature, marker='o')
plt.title('Weekly Temperature')
plt.xlabel('Days')
plt.ylabel('Temperature (°C)')
plt.show()
```



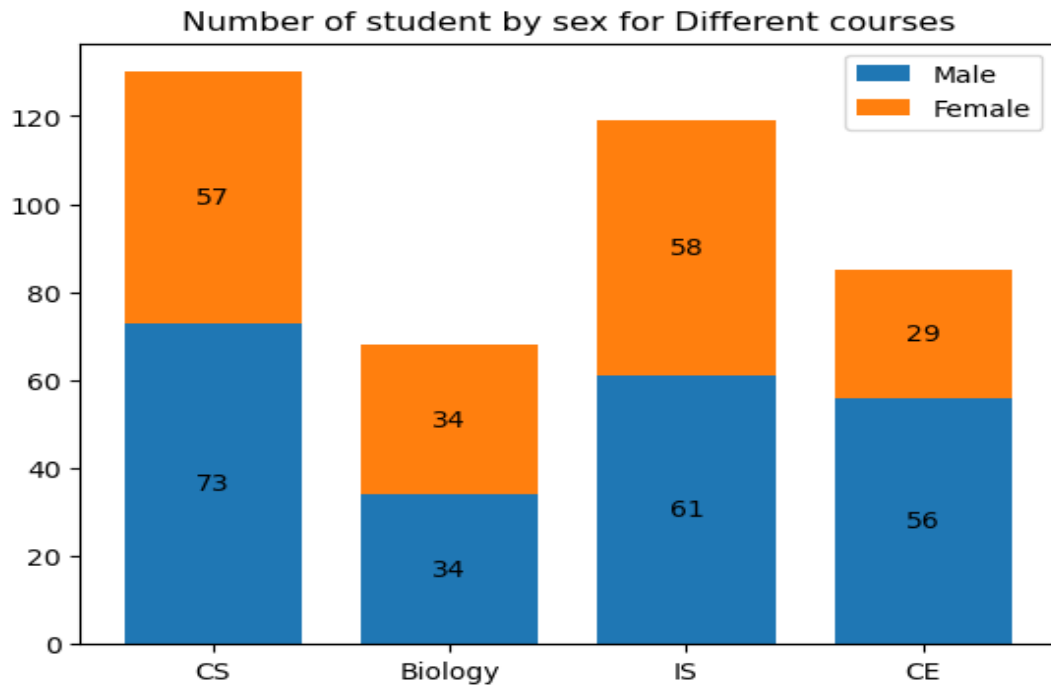
Bar charts

A bar plot (or bar chart) is a graphical representation that uses rectangular bars to compare different categories. The height or length of each bar corresponds to the value it represents. The x-axis typically shows the categories being compared, while the y-axis shows the values associated with those categories. This visual format makes it easy to compare quantities across different groups.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
fruits = ['apple', 'blueberry', 'Avocado', 'orange']
counts = [40, 75, 30, 55]
bar_labels = ['red', 'blue', 'green', 'orange']
bar_colors = ['tab:red', 'tab:blue', 'tab:green', 'tab:orange']
ax.bar(fruits, counts, label=bar_labels, color=bar_colors)
ax.set_ylabel('Supply')
ax.set_title('Fruit by kind and color')
ax.legend(title='Color')
plt.show()
```

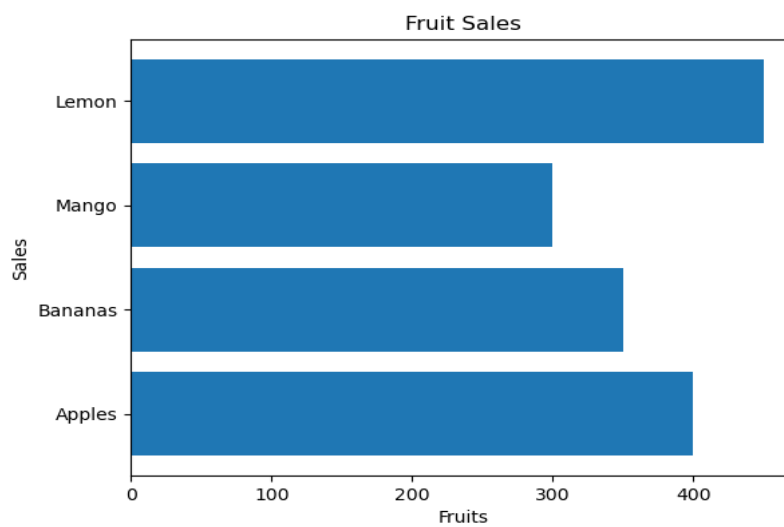


```
import numpy as np
dept = ('CS', 'Biology', 'IS', 'CE')
sex_counts = {'Male': np.array([73, 34, 61, 56]), 'Female': np.array([57, 34, 58, 29])}
width = 0.75 # the width of the bars: can also be len(x) sequence
fig, ax = plt.subplots()
bottom = np.zeros(4)
for sex, sex_count in sex_counts.items():
    p = ax.bar(dept, sex_count, width, label=sex, bottom=bottom)
    bottom += sex_count
    ax.bar_label(p, label_type='center')
ax.set_title('Number of student by sex for different courses')
ax.legend()
plt.show()
```



Horizontal bar-chart

```
import matplotlib.pyplot as plt
import numpy as np
fruits = ['Apples', 'Bananas', 'Mango', 'Lemon']
sales = [400, 350, 300, 450]
plt.barh(fruits, sales)
plt.title('Fruit Sales')
plt.xlabel('Fruits')
plt.ylabel('Sales')
plt.show()
```

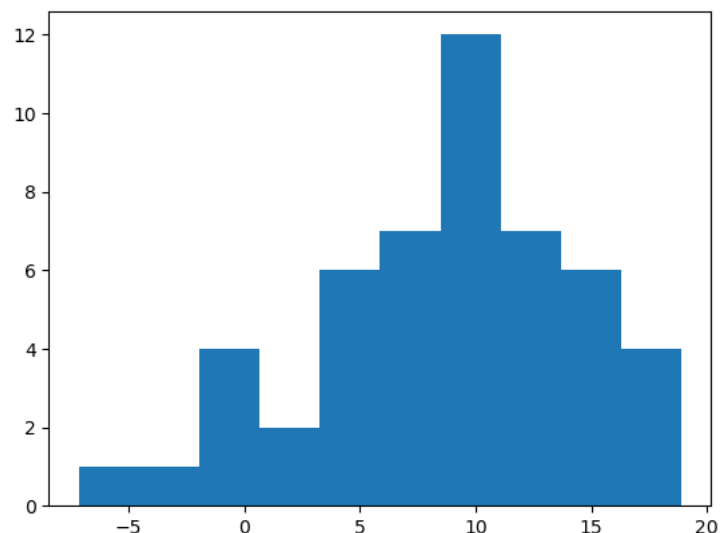


Histograms

A histogram is a graphical representation used in statistics to show the distribution of continuous numerical data. The data is grouped into class intervals (bins), and the height of each bar shows the frequency of values. A histogram helps us understand the shape of the data, such as peaks, spread, and whether the data is symmetric or skewed.

Create a histogram that has a mean value of 10 with 5 standard deviation consisting 50 data.

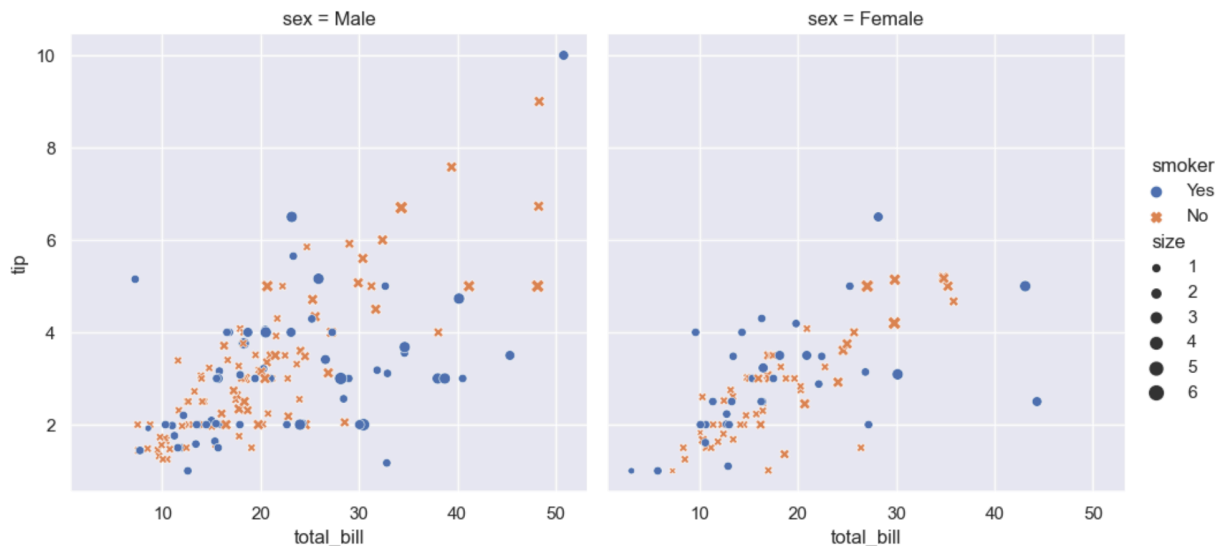
```
import matplotlib.pyplot as plt
import numpy as np
x = np.random.normal(10, 5, 50)
print(x)
plt.hist(x)
plt.show()
```



Seaborn

Seaborn is a Python data visualization library based on matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.

```
# Import seaborn and apply the default theme
import seaborn as sns
sns.set_theme()
tips = sns.load_dataset("tips") # Load an example dataset
# Create a visualization
sns.relplot(data=tips, x="total_bill", y="tip", col="sex", hue="smoker", style="smoker", size="size",)
```

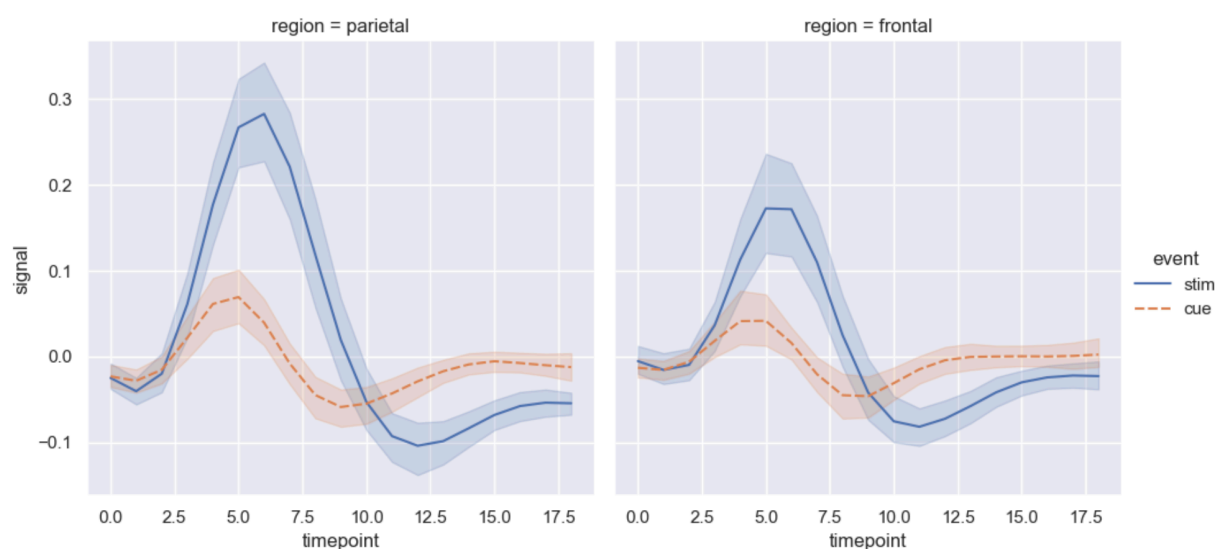


By changing the col value in the `sns.relplot()` you can look at the different plotting

Statistical estimation

Often, we are interested in the *average* value of one variable as a function of other variables. Many seaborn functions will automatically perform the statistical estimation that is necessary to answer these questions:

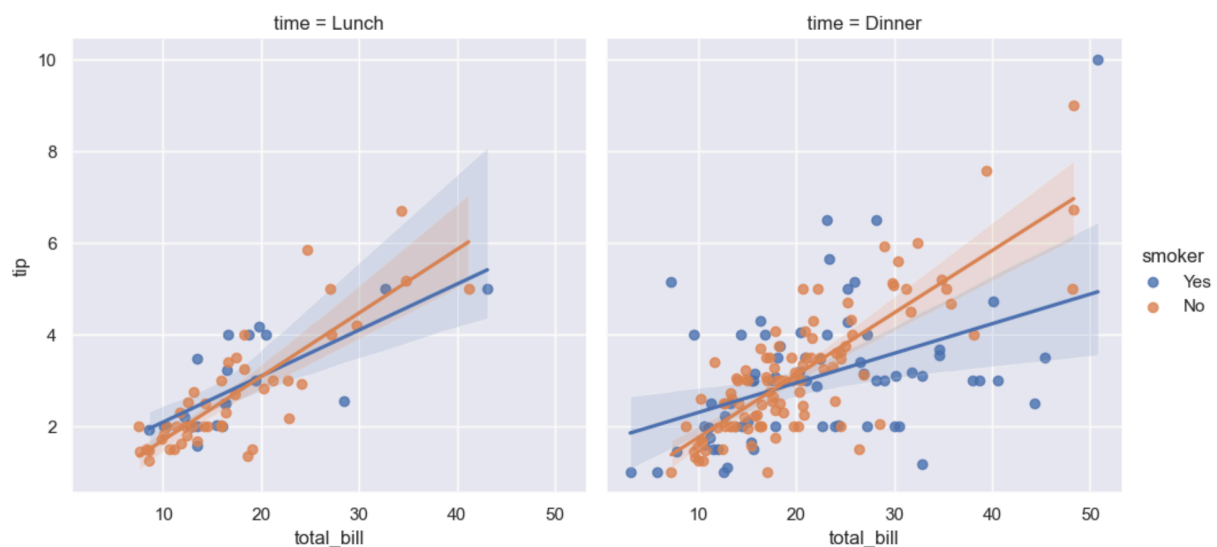
```
fmri = sns.load_dataset("fmri")
sns.relplot(
    data=fmri, kind="line", x="timepoint", y="signal", col="region",
    hue="event", style="event",
)
```



When statistical values are estimated, seaborn will use bootstrapping to compute confidence intervals and draw error bars representing the uncertainty of the estimate.

Statistical estimation in seaborn goes beyond descriptive statistics. For example, it is possible to enhance a scatterplot by including a linear regression model (and its uncertainty) using `lmplot()`:

```
# Import seaborn and apply the default theme
import seaborn as sns
sns.set_theme()
tips = sns.load_dataset("tips") # Load an example dataset
sns.lmplot(data=tips, x="total_bill", y="tip", col="time", hue="smoker")
```



Plotly (Interactive)

Plotly is an open-source library for creating interactive, publication-quality visualizations in Python. Plotly is the home of beautiful, interactive data visualization. Connect to data, build with AI agents, and share your analytics everywhere.

Key Features and Usage Examples

- **Interactive Charts:** Creates interactive scatter plots, line plots, histograms, and bar charts.
- **Scientific & Financial Graphs:** Supports specialized charts such as heatmaps, polar charts, and 3D graphs.
- **AI-Powered Apps:** Plotly Studio allows creating data apps using AI.
- **Data Science Tool:** Commonly used within Jupyter notebooks to display interactive visualizations.

```
# Load the pandas package and create the dataset
import pandas as pd
data = {
    'Country' : ['South Korea', 'China', 'Canada', 'South Korea', 'China',
                'Canada', 'South Korea', 'China', 'Canada'],
```

```

    'Medal' : ['gold', 'gold', 'gold', 'silver', 'silver', 'silver', 'bronze', 'bronze', 'bronze'],
    'Count' : [24, 10, 9, 13, 15, 12, 11, 8, 12]
}
df = pd.DataFrame(data)
df

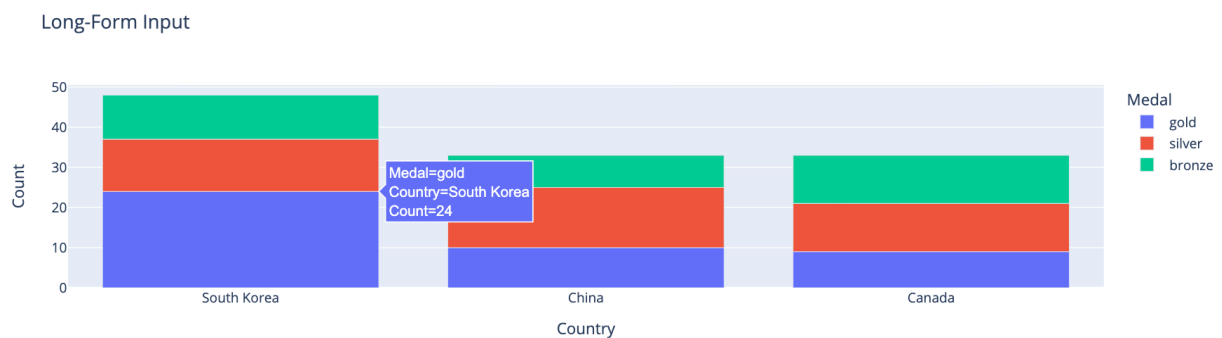
```

	Country	Medal	Count
0	South Korea	gold	24
1	China	gold	10
2	Canada	gold	9
3	South Korea	silver	13
4	China	silver	15
5	Canada	silver	12
6	South Korea	bronze	11
7	China	bronze	8
8	Canada	bronze	12

```

import plotly.express as px
#long_df = px.data.medals_long()
fig = px.bar(df, x="Country", y="Count", color="Medal", title="Long-Form Input")
fig.show()

```



Dash

Dash is an open-source Python framework used to build interactive, web-based analytical applications and dashboards. Developed by Plotly, it allows data scientists to create full-stack web apps using only Python, without requiring deep knowledge of HTML, CSS, or JavaScript.

```

from dash import Dash, dcc, html, Input, Output

```



```

import plotly.express as px

app = Dash(__name__)

app.layout = html.Div([
    html.H1('Restaurant tips by day of week'),
    dcc.Dropdown(
        id="dropdown",
        options=['Mon', "Fri", "Sat", "Sun"],
        value="Fri",
        clearable=False,
    ),
    dcc.Graph(id="graph"),
])

@app.callback(
    Output("graph", "figure"),
    Input("dropdown", "value"))
def update_bar_chart(day):
    df = px.data.tips() # replace with your own data source
    mask = df["day"] == day
    fig = px.bar(df[mask], x="sex", y="total_bill", color="smoker", barmode="group")
    return fig

app.run(debug=True)

```

Restaurant tips by day of week

